

COMP1531

Software Engineering Fundamentals

Week 9

Agile Software Engineering

In This Lecture

— — —

- What is Agile?
- Agile Practises
- Why Agile Works?
- Some Examples

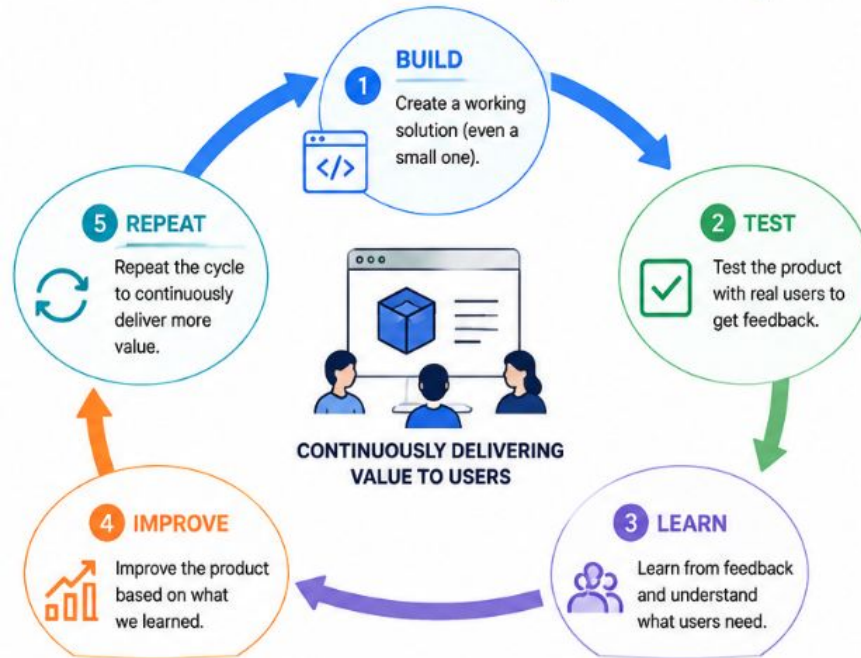
What is Agile?

— — —

- Agile is a software development philosophy that focuses on delivering valuable software through **small, frequent releases** rather than one large release at the end of a project.
- Unlike traditional approaches, Agile assumes that **requirements will change** as customers learn more about their needs.
- The goal is **not simply writing code**, but continuously delivering value to users.

KEY IDEA

Build → Test → Learn → Improve → Repeat



Traditional Approach

One large release at the end



Agile Approach

Small, frequent releases



Why Agile?

In the 1990s many software projects failed because teams spent months **gathering requirements, writing documentation,** and **producing detailed designs** before writing code.

By the time software was delivered:

- customer requirements had changed
- competitors had released better products
- bugs were discovered too late
- developers had built features users didn't actually need

The Response: Agile

Developers realised that instead of spending months planning, it was better to:

- build small pieces of working software
- release frequently
- gather customer feedback early
- adapt to changing requirements
- continuously improve the product

These ideas eventually led to the **Agile Manifesto**, published in **2001**, which formalised many practices that developers had already begun adopting.

The 12 Guiding Principles based on the Agile Manifesto

1. Satisfy customers through early delivery.
2. Welcome changing requirements.
3. Deliver software frequently.
4. Developers and customers work together.
5. Build projects around motivated people.
6. Face-to-face conversations are the most effective way to share information
7. Working software is the primary measure of progress.
8. Maintain sustainable development.
9. Pursue technical excellence.
10. Keep solutions simple.
11. Self-organising teams produce the best designs.
12. Regularly reflect and improve.

What are the 4 Agile Values?

1. Individuals and interactions over processes and tools
2. Working software over comprehensive documentation
3. Customer collaboration over contract negotiation
4. Responding to change over following a plan.

These values prioritise people, collaboration, and adaptability in software development.

— — —

Waterfall Philosophy

"Plan everything first."

Agile Philosophy

"Build something useful today, then improve it tomorrow."

Agile Practices

Successful Agile teams commonly use:

User Stories

"As a student,
I want to reset my password
so that I can access my account."

Test Driven Development

Write tests before writing
code.

Continuous Integration

Merge code frequently.

Automatically run tests after every
change.

Refactoring

Improve code without changing
behaviour.

Keeps software maintainable.



PASSWORD RESET FEATURE

1



USER STORY



As a registered user,
I want to **reset my password**
so that I can regain access to
my account if I forget it.

Acceptance Criteria

- ✓ User enters email
- ✓ Reset email sent
- ✓ Reset link expires in 30 minutes
- ✓ User creates a new password

2



TEST-DRIVEN DEVELOPMENT (Write Tests First)



- ✓ Valid email sends reset link
- ✓ Unknown email is rejected
- ✓ Reset token expires after 30 minutes
- ✓ Password updates successfully
- ✓ User can login with new password



Only after these tests are written do we implement the code.

3



CONTINUOUS INTEGRATION (Every Git Push)



Developer
Pushes Code



GitHub Actions
Starts



Run
Unit Tests



Run
Integration Tests



Build
Application



Tests Pass
Merge to Main



Tests Fail
Merge Blocked

4



REFACTORING

BEFORE

```
if (email !== '' &&  
    email.includes('@') &&  
    users[email]) {  
  ...  
}
```



AFTER

```
if (isValidEmail(email) &&  
    userExists(email)) {  
  ...  
}
```

Benefits

- ✓ Easier to read
- ✓ Easier to maintain
- ✓ Reusable helper functions
- ✓ Same behaviour



DEPLOY NEW PASSWORD RESET FEATURE



Real-world example

Imagine Uber developing a new "Split Fare" feature.

Waterfall

- Spend 8 months designing everything
- Release once
- Discover customers wanted automatic splitting

Agile

- Build a simple version in 2 weeks
- Release to 5% of users
- Gather feedback
- Improve every sprint

COMP1531 Example

Instead of waiting until Week 10 to demonstrate your project, you build

- **Iteration 1**
- receive tutor feedback -> Improve
- **Week 8 Check-in**
- receive tutor feedback -> Improve
- deliver **Iteration 2**

Is this Agile development. Why?

Why Agile Works?

Agile succeeds because it shortens the feedback loop.

Traditional development

- Idea -> 12 months -> Release -> Feedback

Agile

- Idea -> Release -> Feedback -> Improve - Release Again

Fast feedback means fewer mistakes.

Challenges of Agile

— — —

Agile is powerful but not perfect.

Challenges include:

- changing requirements
- estimating effort
- maintaining documentation
- communication between distributed teams
- balancing speed with quality

Agile requires

- trust
- communication
- discipline

rather than simply following ceremonies.

Common Misconceptions

Myth 1

Agile means no planning.

Reality:

Agile plans continuously.

Myth 2

Agile means no documentation.

Reality:

Documentation still matters.

*Only unnecessary documentation
is removed.*

Common Misconceptions

Myth 3

Scrum = Agile.

Reality:

*Scrum is only one Agile
framework.*

Myth 4

Agile means working faster.

Reality:

*Agile means delivering value
sooner.*

Agile vs Waterfall

--- Waterfall

- One large release
- Heavy documentation
- Requirements fixed
- Testing at end
- Customer involved at start
- Difficult to change

Agile

- Many small releases
- Working software
- Requirements evolve
- Continuous testing
- Customer involved continuously
- Easy to adapt

— — —

Example

Netflix does **not** wait one year before releasing improvements.

Instead, small updates are released every few days.

What is the difference between Agile, Scrum, Kanban, and XP?

- Agile is a set of values and principles for software development, while Scrum, Kanban, and Extreme Programming (XP) are specific frameworks that implement Agile principles.
- **Scrum** uses time-boxed sprints and defined roles, **Kanban** focuses on visualizing work and limiting work in progress, and **XP** emphasizes technical excellence and frequent releases.

— — —

 Features	 Scrum	 Kanban	 Extreme Programming (XP)
 Framework Type	Iterative & Incremental	Flow-based	Iterative with strong engineering practices
 Team Roles	Scrum Master, Product Owner, Development Team	No defined roles (flexible)	XP Coach, Developers, Customer
 Planning Approach	Time-boxed sprints (usually 2–4 weeks)	Continuous flow, no fixed timeframes	Iterations with planning and frequent releases
 Work Visualization	Sprint Backlog & Burndown Chart	Kanban Board (To Do, Doing, Done)	Task cards, test lists, etc.
 Changes During Execution	Not allowed mid-sprint	Allowed anytime	Allowed within iteration
 Customer Involvement	Primarily at sprint reviews	Ongoing feedback encouraged	High customer is part of the team
 Delivery Frequency	End of each sprint	Continuous	Every 1–2 weeks
 Core Focus	Delivering prioritized features incrementally	Visualizing and optimizing workflow	Technical excellence, rapid feedback
 Ideal Use Case	Projects with changing requirements, team-driven	Operations/support work, continuous delivery	High-risk tech projects needing code quality

Key Takeaways

- ✓ Agile is a philosophy, not a process.
- ✓ The Agile Manifesto values people, collaboration and adaptability.
- ✓ Agile delivers software in small increments.
- ✓ Scrum, Kanban and XP are frameworks that implement Agile.
- ✓ Customer feedback drives development.
- ✓ Continuous improvement is central to Agile.
- ✓ Successful Agile teams focus on delivering value rather than simply following a plan

Feedback

— — —



Or go to the [form here](#)